Orientação a Objetos

Classes, composição e propriedades

CLASSES E INSTÂNCIAS

Definir
`class Produto:`

Construtor
`def __init__(self, nome):`

Estado
`self.nome = nome`

Método
`def exibir(self):`

Instanciar
`p = Produto("Livro")`

Inspecionar
`isinstance(p, Produto)`

ENCAPSULAMENTO

Invariante
Estado sempre válido

Atributo interno
`_atributo` (convenção)

Property
`@property`

Setters
`@atributo.setter`

Validação
`raise ValueError(...)`

COMPOSIÇÃO

Relação
Tem-um: Carrinho tem itens

Armazenar
`self.itens = []`

Delegar
`item.calcular()`

Vantagem
Reduz acoplamento

HERANÇA

Relação
É-um: Cachorro é Animal

Declarar
`class Filho(Pai):`

Super
`super().__init__()`

Polimorfismo
Contrato mantido

Risco
Evitar herança oportunista

DATACLASSES

Importar
`from dataclasses import dataclass`

Decorator
`@dataclass`

Campos
`nome: str`

Validação
`def __post_init__(self):`

Atenção
Type hints não validam tipos

TESTES E DESIGN

Teste
`assert obj.total == 10`

Dependências
Injetar em vez de criar internamente

Manutenção
Preferir responsabilidade única

Composição
Escolha padrão quando possível

Representação
`__repr__` / `__str__`